

Apresentação — o código do site da Barbearia Magnum

Roteiro textual para 5 pessoas (3 de programação, 2 de documentação). Cada tópico abaixo é de uma pessoa; cada sub-título é um slide. Tudo o que está escrito aqui existe no projeto (`~/magno → /mnt/d/projetos_wsl/magno`) — números conferidos no dia da montagem.

Ordem sugerida: T1 → T2 → T3 → T4 → T5. ~5 min por pessoa (3 slides + 1 demo/print).

TÓPICO 1 — Pessoa 1 (programação)

"Um servidor só serve o site e o sistema"

Slide 1.1 — FastAPI + SQLite na mesma origem (sem CORS, sem build)

- `server/main.py` (701 linhas) é um único app FastAPI: responde a API em `/api/...` e entrega as páginas de web/ (`app.mount("/", StaticFiles(...))`). Por isso não existe CORS: cliente e servidor são o mesmo endereço (D18).
- Páginas servidas pelo próprio backend: `/` (home), `/login` → `entrar.html`, `/perfil` → `perfil.html`, `/account` → `conta.html`, `status.html`.
- Sobe com `uvicorn` em `127.0.0.1:8100` e roda como serviço (`systemd --user: magno-server`).
- Banco SQLite no ext4 do WSL (`~/local/share/magno/magno.db`), **não** na pasta do projeto: gravar em `/mnt/d` (DrvFs/9p) é centenas de vezes mais lento — medido, 238x (decisão D21). O backup diário é que copia o `.db` para a pasta do projeto.
- Publicação: Cloudflare Tunnel (`magno-tunnel`) + `magnum.autoava.us`.

Slide 1.2 — O health-check conta o estado do sistema em uma chamada

- `GET /api/saude` devolve: versão, hora UTC, estado do banco, quais logins estão ligados e as regras de negócio **lidas do banco**.
- Resposta real de agora: `ok: true`, 13 tabelas, 11 índices, integridade: `ok`, login por telefone+PIN ligado / e-mail com código ligado / Google ligado (em modo de teste), regras `slot 30 min · buffer 5 min · antecedência 1 h · janela 60 dias · cancelar até 2 h antes`.
- Tela pronta para mostrar: `/status.html` consome esse JSON e imprime tudo na página.

Slide 1.3 — Front-end sem build: SPA vanilla com CSP estrita

- HTML + CSS + JS puros em web/ (`styles.css` tem 1158 linhas), zero dependência de build (D13/D18). O que o navegador baixa é exatamente o que está no repositório.
- Design system único com variáveis CSS (`--latao`, `--papel`, `--tinta`, `--fundo`) e duas composições da mesma home: `index.html` (editorial) e `index-b.html` (placa).
- A CSP é estrita (`script-src 'self'`): **nenhum** handler inline — todo comportamento está em arquivo externo (`app.js`, `auth.js`, `conta.js`, `perfil.js`, `turnstile.js`).
- Mobile-first: alvos de toque ≥ 44 px, campos de 16 px (evita o zoom automático do iOS), menu hambúrguer igual em todas as páginas — auditado por script em 6 páginas × 4 larguras.

Mostrar na tela: `http://127.0.0.1:8100/status.html` (health-check ao vivo) e o `styles.css` (bloco de variáveis no topo).

TÓPICO 2 — Pessoa 2 (programação)

"Como o cliente entra: três portas de login e uma sessão por token"

Slide 2.1 — Três formas de entrar (uma delas é a principal)

- **E-mail + código de 5 dígitos** (entrada principal, D23): POST /api/auth/codigo envia, POST /api/auth/verificar confere. O código vale 10 min, é de uso único, aceita no máximo 5 tentativas e o banco guarda só o hash dele.
- **Google** (D22): OAuth 2.0 authorization code no servidor. O state é de uso único (10 min), o segredo do cliente nunca vai ao navegador e uma conta que já existe com o mesmo e-mail é **vinculada**, não duplicada.
- **Telefone + PIN de 4 a 6 dígitos** (D16, para o balcão): PINs óbvios são recusados no cadastro (0000, 1234, sequências, pedaço do próprio telefone).
- Senha existe, mas é **atalho opcional** (POST /api/auth/senha) — ninguém é obrigado a criar.
- Tudo isso é o server/auth.py (423 linhas) + as rotas em server/main.py.

Slide 2.2 — A sessão é um token opaco, não um JWT

- secrets.token_urlsafe(32); o banco guarda **apenas** sha256(token) na tabela sessoes, com validade de 24 h — token roubado do banco não é utilizável.
- Logout revoga de verdade (revogada_em): o mesmo token usado de novo devolve 401. POST /api/auth/logout-all derruba todas as sessões da conta (medida de recuperação).
- O HTML da área logada **não tem dado pessoal**: conta.js só libera a tela depois de validar o Bearer em /api/account; sem token, volta para o login.
- O token vive no sessionStorage (some ao fechar a aba) e não é cookie.

Slide 2.3 — O que protege a entrada

- PIN/senha nunca em claro: pbkdf2_sha256 com sal e 200.000 iterações.
- Lockout de 5 falhas / 15 min por telefone+IP, e a mensagem de erro é **sempre a mesma** — não revela se o telefone existe.
- Limite por IP por ação (server/limites.py): 5 contas/hora, 6 códigos/hora, 15 logins/15 min, 12 agendamentos/hora e um teto geral de 600 requisições/min.
- Cabeçalhos de segurança em toda resposta: nosniff, X-Frame-Options: DENY, Referrer-Policy: no-referrer, HSTS atrás do túnel, CSP; corpo acima de 1 MB → 413; SQL 100% parametrizado.
- Cloudflare Turnstile está **pronto mas desligado** (D14): a chave é amarrada ao domínio, então só liga quando o endereço definitivo estiver fixo.

Mostrar na tela: /login (fluxo e-mail → código) e o trecho de auth.py que grava sha256(token).

TÓPICO 3 — Pessoa 3 (programação)

"Agendar sem sobrepor: o coração do sistema"

Slide 3.1 — Um agendamento pertence a um profissional

- POST /api/bookings exige Bearer, valida o telefone no padrão E.164, converte data/hora local (America/Sao_Paulo) para UTC e recusa horário que já passou (400 horario_passado).
- Só aceita serviço + barbeiro existentes e habilitados entre si (servico_profissional).
- Guarda **snapshots** de duração e preço no momento da reserva: mudar o preço do serviço depois não reescreve o passado.
- O campo fim já inclui o buffer de 5 min: o cliente vê 10:00-10:30, a agenda reserva 10:00-10:35 (D10).
- Resultado devolvido: código público do agendamento (mg-...) — é ele que aparece no comprovante e na agenda.

Slide 3.2 — Duas defesas contra dois clientes no mesmo horário

- Defesa 1 (código): consulta de sobreposição dentro da transação — `inicio < :fim_novo AND fim > :inicio_novo` entre os status ativos → resposta 409 `horario_ocupado` com a mensagem "Esse horário acabou de ser ocupado. Escolha outro."
- Defesa 2 (banco): índice único **parcial** em (`profissional_id`, `inicio`) para status ativos — é a trava dura, vale mesmo com duas requisições simultâneas (D9).
- Detalhe que já deu erro e foi corrigido: todo `except IntegrityError` faz `rollback()` antes de responder 409 — sem isso a conexão continuava segurando o lock e a outra recebia `database is locked`.
- Prova: o script de validação do schema roda o cenário de **duas threads no mesmo slot** → uma escrita cria, a outra é recusada, e nenhum horário duplicado fica no banco.

Slide 3.3 — O banco que sustenta isso (13 tabelas)

- `usuarios`, `codigos_email`, `profissionais`, `servicos`, `servico_profissional`, `horarios`, `excecoes`, `bloqueios`, `agendamentos`, `sessoes`, `logins_pendentes`, `eventos`, `configuracoes` — criadas por `docs/schema.sql`, com `IF NOT EXISTS` e `seed INSERT OR IGNORE` (subir o servidor N vezes não duplica nada).
- Tempo: tudo gravado em UTC ISO-8601 (2026-09-15T13:00:00Z) e exibido em São Paulo — conversão só na borda (D4/D11). Dinheiro em centavos, inteiro (D5). Telefone é o identificador do cliente, único (D6).
- Cancelamento **não apaga**: muda status + motivo. Exclusão definitiva de atendimento futuro é só do admin e gera evento. A tabela `eventos` é a auditoria — responde "quem apagou o atendimento das 15h?".
- As regras (slot 30 min, buffer 5, antecedência 1 h, janela 60 dias, cancelar até 2 h antes) vivem na tabela `configuracoes` e são editáveis — **nenhuma delas está escrita no código**.

Mostrar na tela: /account agendando um horário e, em seguida, o mesmo horário de novo (o 409 na tela). Depois `sqlite3 ... "select * from agendamentos"`.

TÓPICO 4 — Pessoa 4 (documentação)

"A documentação é a fonte da verdade — e ela dirige o código"

Slide 4.1 — Antes da primeira linha de código, o projeto escreveu o plano

- docs/00-PLAN0.md: qual problema existe (a barbearia agenda por WhatsApp, na mão: buraco na agenda, conflito de horário e nenhum histórico), o objetivo, a métrica de sucesso (**agendar em menos de 60 s pelo celular**), atores e permissões (cliente / barbeiro / admin), regras de negócio, arquitetura, telas, riscos e fases.
- O plano também escreve o que **NÃO** entra na v1: pagamento online, fidelidade, estoque, comissão, múltiplas unidades, app nativo. Escopo fechado por escrito é o que protege o prazo.
- Ponto de destaque para a banca: a documentação é a razão de o sistema não ter virado um amontoado de telas soltas.

Slide 4.2 — Decisões numeradas, com consequência técnica (D1 a D25)

- docs/03-DECIS0ES.md registra cada decisão como **pergunta → escolha → consequência**, e não como opinião. Exemplos: D16 (telefone+PIN), D22 (Google OAuth server-side), D23 (código por e-mail em vez de SMS pago), D24 (provedor de e-mail plugável), D25 (primeiro acesso pergunta nome e depois idade).
- Exemplo com número na mão: a D21 nasceu de uma medição — 200 gravações em SQLite custaram 0,002 s no ext4 e 0,462 s na pasta montada do Windows: **238x mais devagar**. Decisão: o banco fica no ext4, o backup no D:.
- O que ainda está em aberto também fica escrito (dados reais da loja, quem é o admin, a grafia "Magno" × domínio "magnum") — nada fica só na cabeça de alguém.

Slide 4.3 — Contrato, schema executável e backlog por fase

- docs/02-API.md: cada rota com payload, resposta e código de erro padronizado (400 validacao, 401 sessao_invalida, 403 prazo_excedido, 409 slot_ocupado, 413 corpo_grande, 429 rate_limit). É o documento que front-end e back-end usam juntos.
- docs/schema.sql é **executável**: é dele que o banco nasce (tabelas, índices, CHECKs, seed) e também a referência do modelo de dados (docs/01-MODELO-DE-DADOS.md).
- docs/04-BACKLOG.md organiza o trabalho em fases F0→F7, cada tarefa com **critério de pronto**: F0 (estrutura/banco) e F1 (auth/segurança) fechadas; F2, F4, F6 e F7 com boa parte entregue.
- docs/05-DEPLOY.md e docs/06-LOGIN.md guardam o passo a passo operacional — inclusive o erro exato do túnel (https na origem em vez de http) e a configuração de e-mail (SPF/DKIM).

Mostrar na tela: abrir as 4 pastas/arquivos de docs/ e o schema.sql rodando (o banco nasce dele).

TÓPICO 5 — Pessoa 5 (documentação)

"Como o projeto prova que funciona — e o que ainda não está pronto"

Slide 5.1 — Verificação em quatro camadas

- **150 testes de servidor** (pytest, banco descartável em /tmp, nunca o banco de verdade): auth, Bearer, código por e-mail, Google, limites, migração de banco e smoke. Conferido agora: 150 passed.
- **Verificação ao vivo na API** (scripts/testa_bearer.py): 34 checagens em 8 blocos — cadastro, login, sessão, agendamento real, **409 no mesmo horário**, listagem da conta e revogação de token. Conferido agora: 34 OK, 0 falhas.
- **Verificação no navegador** (Playwright): /perfil 24 checagens, login 22, login por e-mail 18, home dinâmica 26, identidade de movimento 15.
- **Auditoria de celular**: 6 páginas × 4 larguras (360/390/414/768) medindo rolagem horizontal, estouros, alvos < 44 px, texto < 12 px e erros de console — critério de pronto: 0 problemas.
- **Provas visuais**: 31 prints em docs/provas/ (primeiro acesso, agenda com o nome de quem entrou, login por e-mail, home desktop e mobile).

Slide 5.2 — Privacidade e segurança tratadas como requisito (LGPD)

- Coleta o mínimo: nome, telefone e e-mail opcional; o consentimento é gravado com data (consentimento_lgpd_em) e é obrigatório no cadastro.
- Nenhum dado pessoal no HTML: o Bearer vive no sessionStorage e tudo em /api sai com Cache-Control: no-store.
- Exclusão de conta está prevista no modelo para **anonimizar** o histórico (marca anonimizado_em, remove o telefone) em vez de apagar o registro — relatório continua coerente.
- E-mail transacional com provedor plugável: modo arquivo (não envia nada, grava em logs/emails/) para desenvolver e testar; para valer, precisa de provedor + SPF/DKIM no DNS.

Slide 5.3 — Estado real do site hoje (transparência)

- **Funcionando**: home institucional, login por e-mail + código, login com Google (modo de teste), login por telefone + PIN, primeiro acesso em /perfil (nome → idade) e a agenda do cliente em /account com reserva real de horário.
- **Pendente e assumido**: a grade de dias e horas do cartão de agendamento ainda é **fixa no HTML** (16 a 20/09/2026) — passada essa data não dá mais para agendar. A correção é a fase F3 (motor de disponibilidade, com horarios, excecoes, bloqueios e buffer).
- **Pendente**: painel da loja (F5), remarcar/cancelar pelo cliente (F4), CSV e páginas de privacidade (F6).
- **Falta 1 campo no painel Cloudflare** para o domínio funcionar: trocar o tipo de HTTPS para HTTP no Public Hostname magnum.autoava.us → 127.0.0.1:8100 (é a causa do 502 atual).
- **Nota honesta para quem for responder perguntas**: o script scripts/valida_schema.py hoje acusa 16/17 por causa de uma checagem desatualizada ("11 tabelas" — o schema já tem 13, com codigos_email e logins_pendentes). O problema é o script, não o banco: são 16 verificações OK e a que falha é só a lista de nomes de tabela.

Mostrar na tela: pytest rodando (150 passando) e um print de docs/provas/.

Cola rápida (para quem for responder perguntas)

- Site: ~/magno → /mnt/d/projetos_wsl/magno · porta 8100 · banco em ~/.local/share/magno/magno.db.
- Stack: Python 3.11 + FastAPI + SQLite (SQL puro, sem ORM) + SPA vanilla, sem build.
- Números de hoje: 13 tabelas · 11 índices · 150 testes pytest · 34 checagens ao vivo · 31 prints de prova · 16 commits.
- Comandos: ./scripts/run.sh (subir) · MAGNO_DB=/tmp/test.db ./venv/bin/pytest tests/ -q (suíte) · ./venv/bin/python scripts/testa_bearer.py (API ao vivo) · systemctl --user restart magno-server (reiniciar).